

이중 채널 상태 동기화

Webhook + 폴링 — 단일 경로의 취약점을 어떻게 보완하는가

Webhook

pollStaleRequests

역등성

두 시스템 간 상태 동기화 문제

THE CORE PROBLEM — DISTRIBUTED STATE GAP



핵심 질문 — VASP가 TX를 처리 완료했는데 우리 시스템은 어떻게 알 수 있는가? 그리고 그 알림이 도착하지 않으면?

단일 콜백 방식의 취약점

SINGLE WEBHOOK — SINGLE POINT OF FAILURE



실패 시나리오

- ① 네트워크 단절 → Webhook 미도달
- ② 서버 재시작 → 수신 직후 크래시
- ③ VASP 측 발송 실패 (자체 버그)
- ④ 타임아웃 · 부하 → 큐 밀려서 유실

Webhook 단독의 한계

- 단발성 — 재전송 보장 없음
- 수신 즉시 처리 실패 시 상태 누락
- VASP 파트너사 신뢰도에 전적 의존
- 장애 감지 자체가 불가능

결과

- **PENDING** 상태 영구 고착
- 고객 NFT 미발행 — 서비스 장애
- 재발행 시도 → 이중 발행 위험

결론 — Webhook은 빠른 경로(fast path)이지, 신뢰할 수 있는 유일한 경로가 아니다.

콜백 누락 경로 전체 분석

ALL PATHS WHERE WEBHOOK CAN BE LOST

누락 원인	발생 위치	감지 가능?	영향
네트워크 단절	VASP → KYOBO 구간	불가	PENDING 고착
KYOBO 서버 재시작	수신 직후 처리 전	불가	수신 기록 없음
VASP 발송 버그	VASP 내부	불가	Webhook 미발송
처리 큐 포화	KYOBO 내부	조건부	지연 후 유실 가능
DB 트랜잭션 실패	상태 업데이트 직전	조건부	롤백 → PENDING 유지
파트너사 SLA 미달	VASP 처리 지연	가능	10분 초과 → 폴링 대상

공통 패턴 — 어느 경우든 KYOBO DB에는 **PENDING** 이 남는다. 이 상태가 일정 시간 이상 지속되면 "누락"으로 판단할 수 있다.

비유 — 택배 수령과 문자 알림

ANALOGY — DELIVERY NOTIFICATION

📱 문자 알림 = Webhook

- 택배사가 "배송 완료" 문자 발송
- 빠르고 편하지만 — 가끔 미도달
- 수신 못해도 택배는 이미 도착함
- 문자 하나 놓쳤다고 물건이 사라지진 않음

빠른 경로(fast path) — 대부분 동작, 드물게 누락

🏠 직접 문 열어보기 = 폴링

- 문자 안 와도 주기적으로 현관 확인
- 느리지만 — 반드시 상태를 확인함
- "혹시 왔나?" 직접 조회 = `getTransferStatus()`
- 문자 누락을 완전히 보완

안전망(safety net) — 느리지만 누락 없음

이중 채널 전략

문자 오면 즉시 확인(Webhook) + 안 오면 직접 확인(polling) — 둘 다 같은 결과로 처리

두 가지 동기화 전략

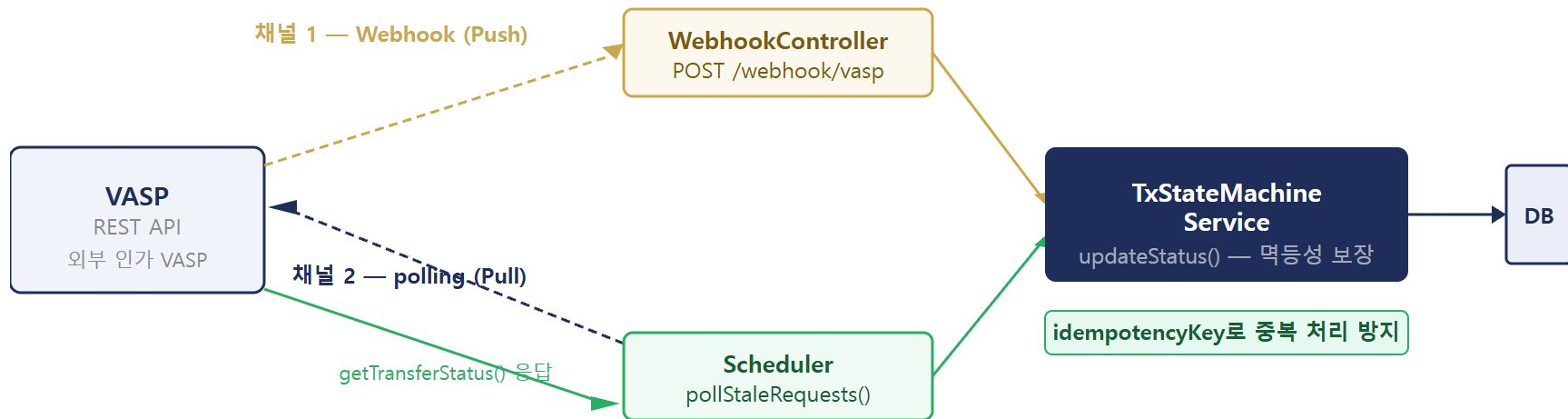
DUAL CHANNEL — WEBHOOK + POLLING

구분	채널 1 — Webhook	채널 2 — pollStaleRequests
트리거	VASP 이벤트 발생 시 즉시	스케줄러 주기 실행 (예: 5분)
방향	VASP → KYOBO (Push)	KYOBO → VASP (Pull)
속도	즉시	지연 (최대 폴링 주기)
신뢰도	단발 — 누락 가능	주기적 — 누락 보완
대상	모든 TX (VASP가 발송 시)	PENDING 10분 초과 TX만
처리 함수	<code>handleWebhook()</code>	<code>pollStaleRequests()</code>
중복 처리	역등성 키로 방지 — 어느 채널이 먼저 도달해도 결과 동일	

설계 원칙 — 두 채널은 경쟁하지 않는다. 먼저 도달한 채널이 처리하고, 나중에 도달한 채널은 역등성으로 무시된다.

전체 아키텍처 — 이중 채널 전체 그림

DUAL CHANNEL ARCHITECTURE



이중 채널이 충돌하지 않는 이유

DUAL CHANNEL — STATE MACHINE PREVENTS DOUBLE TRANSITION

상태 전이 가드

현재 상태가 전이 조건에 맞지 않으면 즉시 return — 중복 전이 불가

```
// TxStateMachineService.ts
async handleMined(id: string, blockNumber: number) {
  const req = await this._getOrThrow(id);
  // Webhook 이미 처리 → PENDING/SUBMITTED 아님 → 스킵
  if (req.status !== 'PENDING' &&
      req.status !== 'SUBMITTED') return;
  await this._transition(req, 'MINED', { blockNumber });
}

async handleConfirmed(id: string) {
  const req = await this._getOrThrow(id);
  // 풀링 이미 처리 → MINED 아님 → 스킵
  if (req.status !== 'MINED') return;
  await this._transition(req, 'CONFIRMED');
}
```

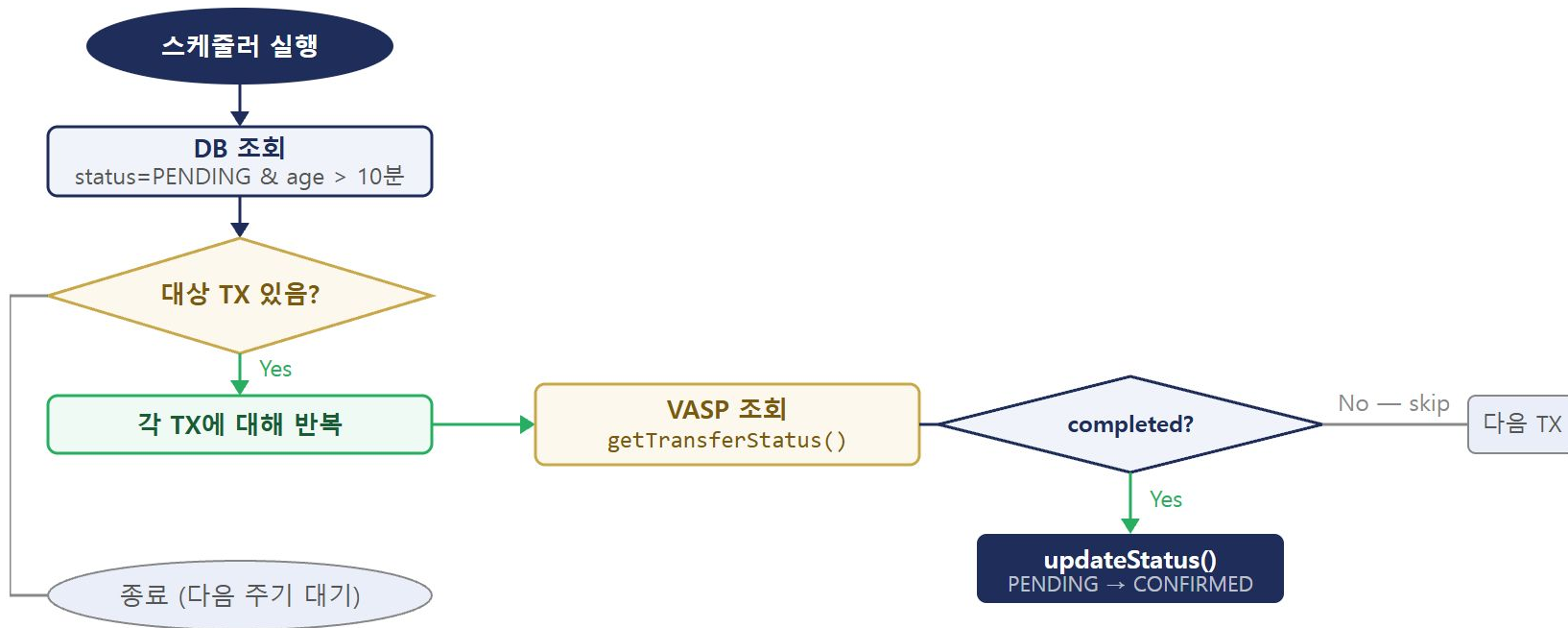
두 채널이 동시 도달하면

- Webhook 먼저 도달 → **CONFIRMED** 저장
- 풀링 나중 도달 → terminal 상태 감지 → 무시
- 풀링 먼저 도달 → **CONFIRMED** 저장
- Webhook 나중 도달 → terminal 상태 감지 → 무시

결론 — 어느 채널이 먼저 도달해도 DB 상태는 한 번만 전이된다. 이중 발행 없음.

pollStaleRequests 알고리즘 설계 (1/2)

POLLING ALGORITHM — FLOWCHART



pollStaleRequests 알고리즘 설계 (2/2)

POLLING TARGET SELECTION — WHY PENDING > 10MIN ONLY

폴링 대상 조건

- 상태: **PENDING** 또는 **SUBMITTED**
- 경과 시간: 최초 제출 후 10분 초과
- 조건 동시 충족 시에만 폴링

10분 임계값의 근거

- 정상 처리 시간: VASP SLA 기준 ~1-3분
- Webhook 지연 여유분: 최대 5분 가정
- 10분 = 정상 범위 벗어난 것이 확실한 시점
- 너무 짧으면 → VASP API 과부하
- 너무 길면 → 고객 대기 시간 증가

왜 모든 PENDING을 폴링하지 않는가

- 방금 제출한 TX도 PENDING — 정상
- 즉시 폴링 = VASP 불필요한 부하
- Webhook이 오는 중일 수도 있음
- 10분 후 = Webhook 포기 선언 시점

```
const stale = await this.repo.findStale({
  status: ['PENDING', 'SUBMITTED'],
  olderThan: new Date(
    Date.now() - 10 * 60 * 1000
  ),
});
```

트레이드오프 — 임계값이 짧을수록 복구 빠름 / VASP 부하 증가. 길수록 안전 / 고객 대기 증가. 파트너사 SLA에 맞춰 조정.

pollStaleRequests 전체 구현

PENDING 10분 초과 → VASP 직접 조회 → 결과별 상태 전이

```
async pollStaleRequests(): Promise<{ processed: number }> {  
  const stale = await this.repo.findPendingOlderThan(STALE_MINUTES); // 10분 초과 PENDING  
  let processed = 0;  
  for (const req of stale) {  
    if (!req.txHash) continue;  
    try {  
      const result = await this.vasp.getStatus(req.txHash);  
      switch (result.status) {  
        case 'confirmed': await this.handleConfirmed(req.id); break;  
        case 'failed':     await this.handleFailed(req.id, result.revertReason); break;  
        case 'not_found':  await this.handleFailed(req.id, 'tx not found'); break;  
        // 'pending' → skip (계속 대기)  
      }  
      processed++;  
    } catch (err) {  
      console.error(`pollStale error ${req.id}`, err); // Bulkhead - 다음 건 계속  
    }  
  }  
  return { processed };  
}
```

① 진입점 — 크론 스케줄러

5분마다 자동 실행 · 수동 호출 없음
findPendingOlderThan(10)
→ 10분 초과 PENDING만 조회

② 결과별 분기

confirmed → handleConfirmed
failed / not_found → handleFailed
pending → skip (다음 주기)

③ Bulkhead — 개별 오류 격리

try/catch = 배치의 격벽
한 건 실패 → 로그 후 다음 건
미처리 건 → 5분 후 재처리

단일 오류가 전체 배치를 중단시키는 문제

Bulkhead(격벽) 패턴 — 개별 실패를 격리해 나머지 처리를 보장한다

pollStaleRequests 실행 흐름



❌ 잘못된 설계 — 오류 전파

[req-050] throw ❌

→ pollStaleRequests 중단

→ [req-051]~[req-100] 미처리

→ 50건 최대 5분 추가 지연

✅ 올바른 설계 — Bulkhead 격리

[req-050] catch → 로그 ⚠️

→ [req-051] 처리 계속 ✅

→ [req-100] 처리 완료 ✅

→ req-050 → 다음 크론 재처리

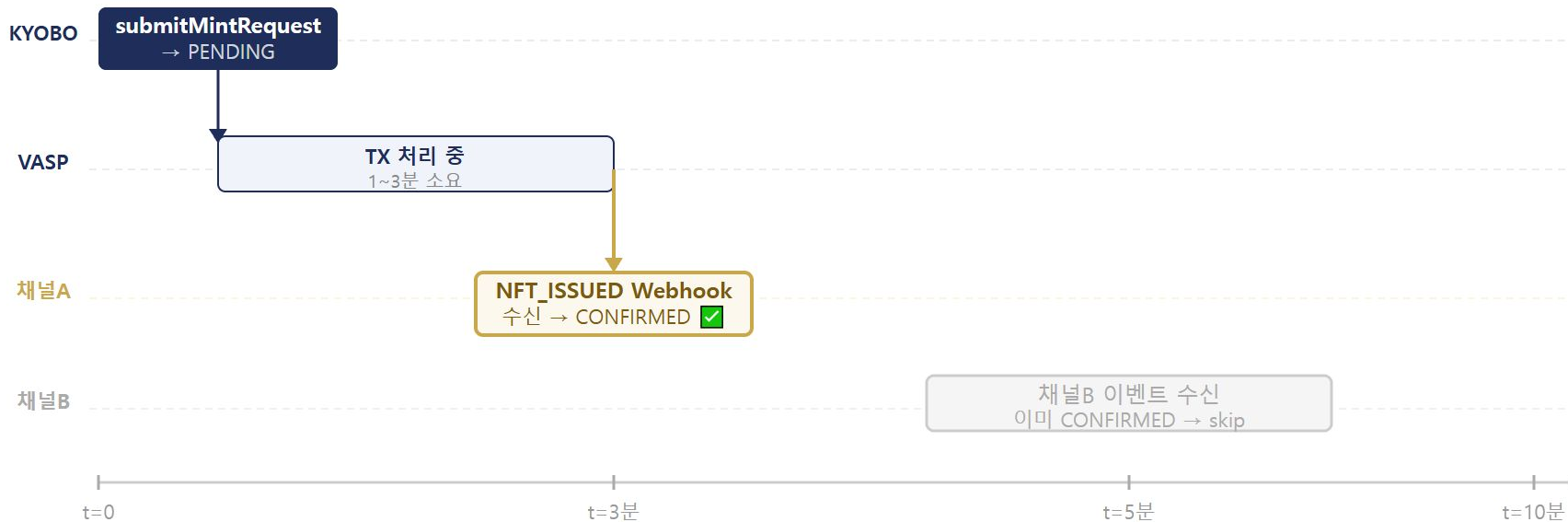
배의 격벽(Bulkhead):

한 구역 구멍 → 전체 침몰 막는다.

개별 try/catch = 배치 루프의 격벽

운영 흐름 ① — 정상 케이스 (채널 A 성공)

HAPPY PATH — WEBHOOK DELIVERS, CHANNEL B SKIPS



채널 A(Webhook)가 가장 빠르다 — 3분 만에 CONFIRMED. 채널 B-풀링은 멍등성으로 skip.

운영 흐름 ② — 채널 A 누락 (Webhook 유실)

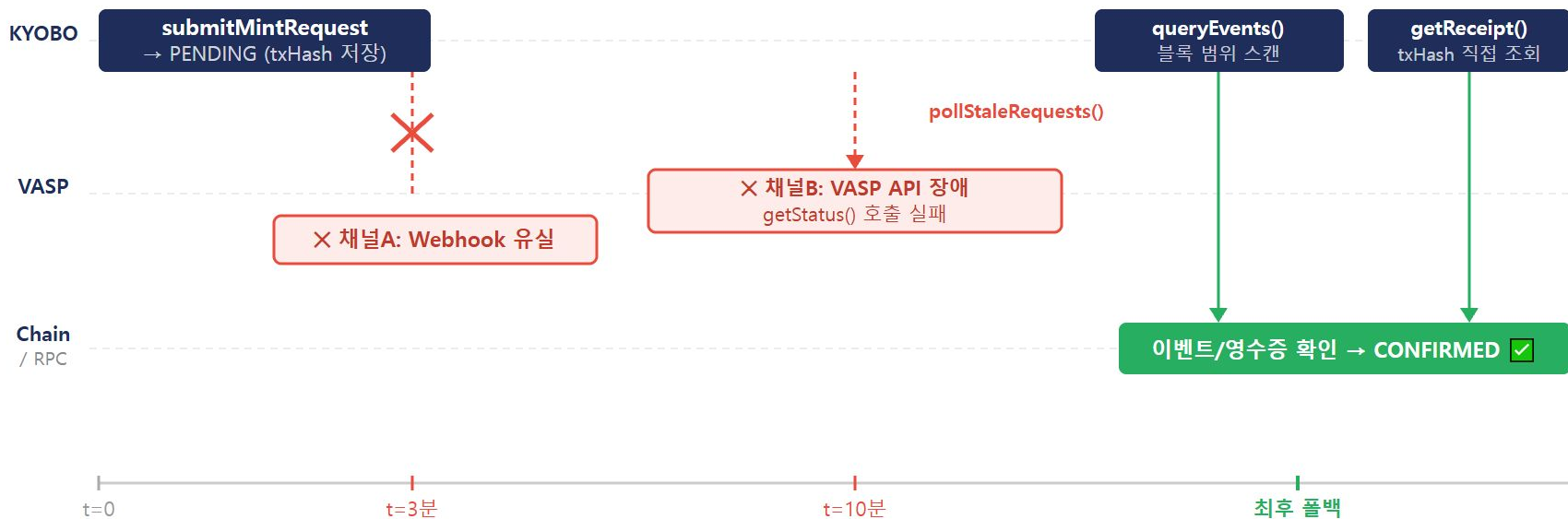
WEBHOOK LOST — CHANNEL B SAVES THE DAY



채널B(`pollStaleRequests`)가 백업 — Webhook 유실 시 10분 내 자동 복구. VASP API 직접 조회 → CONFIRMED 전이.

운영 흐름 ③ — 최악 케이스 (채널 A·B 모두 실패)

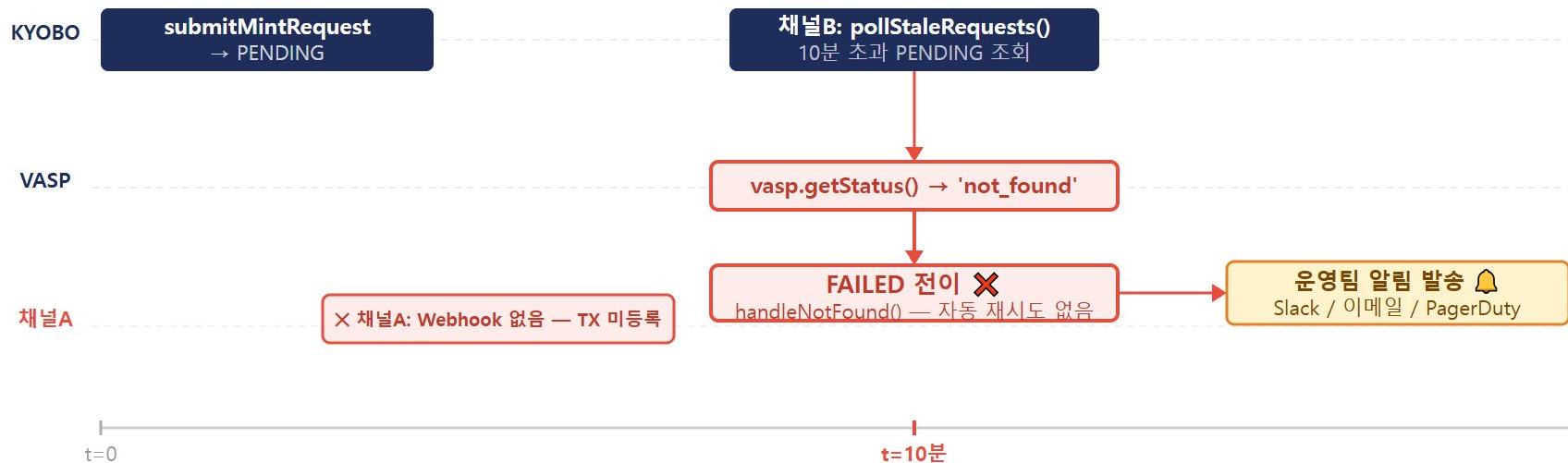
BOTH CHANNELS DOWN — DIRECT CHAIN QUERY IS THE LAST RESORT



3단계 풀백: ① Webhook(채널A) → ② VASP 폴링(채널B) → ③ 블록체인 직접 조회 — queryEvents()(블록 범위 스캔, txHash 불필요) · getReceipt(txHash)(txHash 있을 때) 둘 다 사용 가능. VASP 완전 무관.

운영 흐름 ④ — TX 미등록 (not_found)

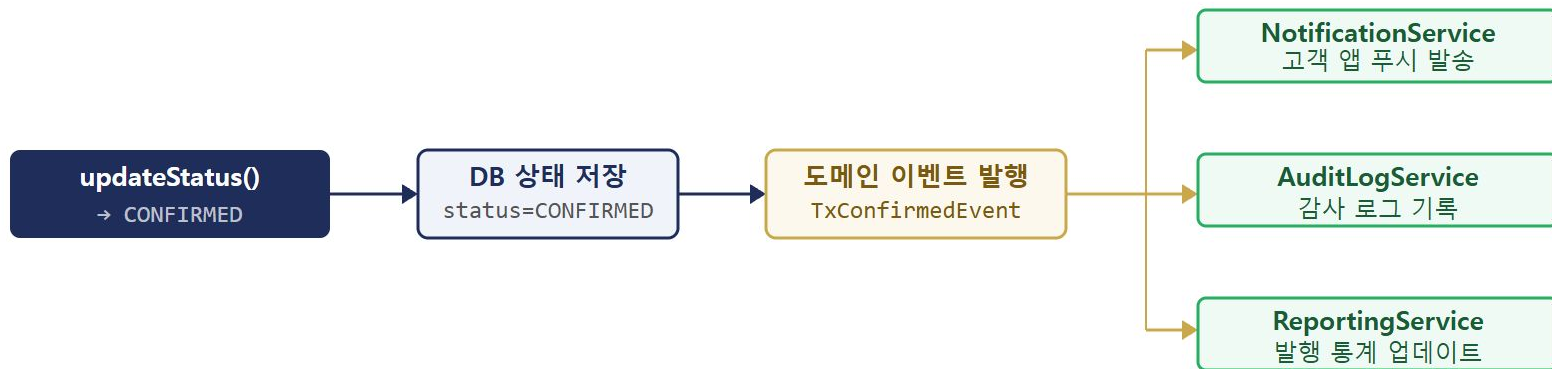
TX NOT IN VASP — FAIL FAST + ALERT OPS TEAM



Fail Fast + Alert — VASP는 살아있지만 TX가 mempool에 없음. 채널A(Webhook) 무응답 → 채널B(pollStaleRequests)가 'not_found' 수신 → FAILED 전이 + 운영팀 수동 개입.

TxStateMachineService 전이 이후 흐름

POST-TRANSITION — WHAT HAPPENS AFTER CONFIRMED



Webhook / 폴링 어느 경로든 동일한 이벤트 발행 → 동일한 후속 처리

핵심 — 수신 채널(Webhook vs 폴링)에 관계없이 TxConfirmedEvent 하나로 통일. 구독자는 채널을 모른다.

폴링을 없애고 Webhook만 쓰면 안 되는가?

WEBHOOK ONLY — WHY IT'S NOT ENOUGH

Webhook만 사용 시 문제

- VASP 미발송 시 KYOBO 감지 불가
- 네트워크 장애 구간 TX 영구 PENDING
- 파트너사 장애 = 서비스 전체 장애
- SLA 보장 수단 없음

단일 실패 지점(SPOF) — VASP Webhook 발송 시스템이 곧 우리 서비스의 신뢰도

폴링이 제공하는 것

- 자기 주도적 상태 확인 — VASP 의존 탈피
- 장시간 PENDING = 이상 감지 트리거
- Webhook 누락의 완전한 보완
- 파트너사 장애 자동 복구

폴링은 느린 대신 자율적 — 외부 시스템에 의존하지 않고 내가 직접 확인한다.

ChainEventListener만 쓰면 안 되는가?

CHAIN EVENT ONLY — CONTEXT & LIMITATION

Phase 1에서 ChainEventListener의 역할

- Phase 1 = VASP에 TX 위탁 — 블록체인 직접 제어 없음
- 온체인 이벤트 구독 가능하나 TX 귀속 판단 복잡
- VASP 내부 상태(completed/failed)와 온체인 상태는 별개
- Phase 3 자체 VASP 시 ChainEventListener 본격 활용

Phase 1 제약 — 우리가 발행한 TX 해시를 VASP로부터 받지 않으면 온체인 이벤트 매핑 불가

Webhook이 제공하는 것

- VASP 처리 완료 즉시 알림 — 가장 빠른 경로
- VASP 내부 상태 + TX 결과 함께 전달
- Phase 1에서 유일한 실시간 채널
- 폴링 주기 이전 즉각 처리 가능

결론 — Phase 1은 Webhook(실시간) + 폴링(안전망)이 최적. Phase 3에서 ChainEventListener 추가.

M3 모듈 완료 기준 체크리스트

S13 ~ S22 전 구현 항목 — 모두 통과해야 M3 완료

상태머신 + 멀티체인 추상화 (S13~S16)

- VALID_TRANSITIONS 맵 — 허용 안 된 전이 → 예외
- submitMintRequest — REQUESTED → SUBMITTED, Idempotency
- IBlockchainAdapter — EVMAAdapter / XRPLAdapter 교체 가능
- EVMAAdapter — mintNFT / subscribeEvents / queryEvents

복구 메커니즘 (S17~S20)

- retryWithBackoff — 지수 백오프 + Jitter, 5회 후 FAILED
- handleTxRevert — FAILED + reason 저장 + 알림
- handleTimeout — gas 20% bump + 새 txHash
- handleReorg — REORGED → 5블록 대기 → 재조회

VASP 추상화 + 이중채널 + 폴링

(S21~S22)

- IVASPAdapter — submit / getStatus / getTransferStatus
- ExternalVASPAdapter — 외부 VASP 연동 + HMAC 검증
- VaspTxClientAdapter — 내부 → 외부 포맷 변환
- 이중 채널 — Webhook + pollStaleRequests로 유실 없음
- Bulkhead — 개별 에러가 배치 중단 안 함
- Idempotency — CONFIRMED 건 재처리 없음

M3 핵심 정리

VASP + TX 상태머신의 3가지 원칙 — 다음 모듈(M4 원장·감사 로그) 브리지

① 상태머신으로 비동기 TX 추적

TX는 즉시 확정되지 않는다

REQUESTED → SUBMITTED → PENDING → MINED → CONFIRMED

각 단계 DB 기록 → 재시작 후에도 복구 가능

② 멀티체인 추상화

비즈니스 로직은 `IBlockchainAdapter`만 안다

체인 교체 = 어댑터만 교체

`EVMAdapter` → `XRPLAdapter` → 상위 코드 변경 없음

③ VASP 추상화

외부 VASP 연동은 `IVASPAdapter`만 안다

`ExternalVASPAdapter` → Phase 3 `KyoboVASPAdapter`

DI 바인딩 1줄 교체로 전환

④ 이중 채널 + 3중 복구로 신뢰성

콜백(빠름, 유실 가능) + 폴링(느림, 확실) = 반드시 처리됨

REVERT → FAILED + reason · TIMEOUT → gas bump · REORG → 5블록 재
조회

다음 모듈 — M4 S23~S26: 내부 원장 + 감사 로그

M3의 CONFIRMED 전이 이후 → **LedgerService**가 `user_nft_holdings`를 갱신하는 방법 · **AuditLogService** SHA-256 체인 봉인 · **ReconcileService**
온체인 대조